

Mobile Application Security Testing Requirements Document

Prepared By: Information Network Security Administration (INSA) 2025
Submitted by: [Client's Name / Organization Placeholder]
Submitted to: Information Network Security Administration (INSA)

Table of Contents

- 1. Introduction
 - 1. Purpose
 - 2. Ownership
 - 3. Scope
- 2. Legal and Administrative Documents
- 3. Technical Documentation for Web Application Security Testing
 - 1. Business Architecture and Design
 - 2. Features of the Web Application
 - 3. Security Functionality Document
 - 4. Secure Coding Standard Documentation
 - 5. Threat Modeling & Risk Management
 - 6. Previous Security Testing Reports
- 4. API Security Audit Requirements

1. Introduction

1.1 Purpose

This document provides an overview of the Web and Mobile Application Security Testing Requirements Document, compiled for the Information Network Security Administration (INSA). The application in scope is the **Deresegn Point of Sale (POS) and SaaS Application**, which integrates with the Ethiopian Ministry of Revenues (MoR) e-invoicing system.

1.2 Ownership

The owner of this document is the **Deresegn Engineering and Compliance Department**.

1.3 Scope

The scope encompasses the security mechanisms, architectural layouts, and data flows of the Deresegn POS/SaaS application:

- The Flutter-based mobile client interfaces.
 - The Laravel 8 (PHP) backend REST API.
 - The local SharedPreferences storage handling offline queuing.
 - The integration with the Ministry of Revenues (MoR) e-invoicing API.
-

2. Legal and Administrative Documents (Mandatory)

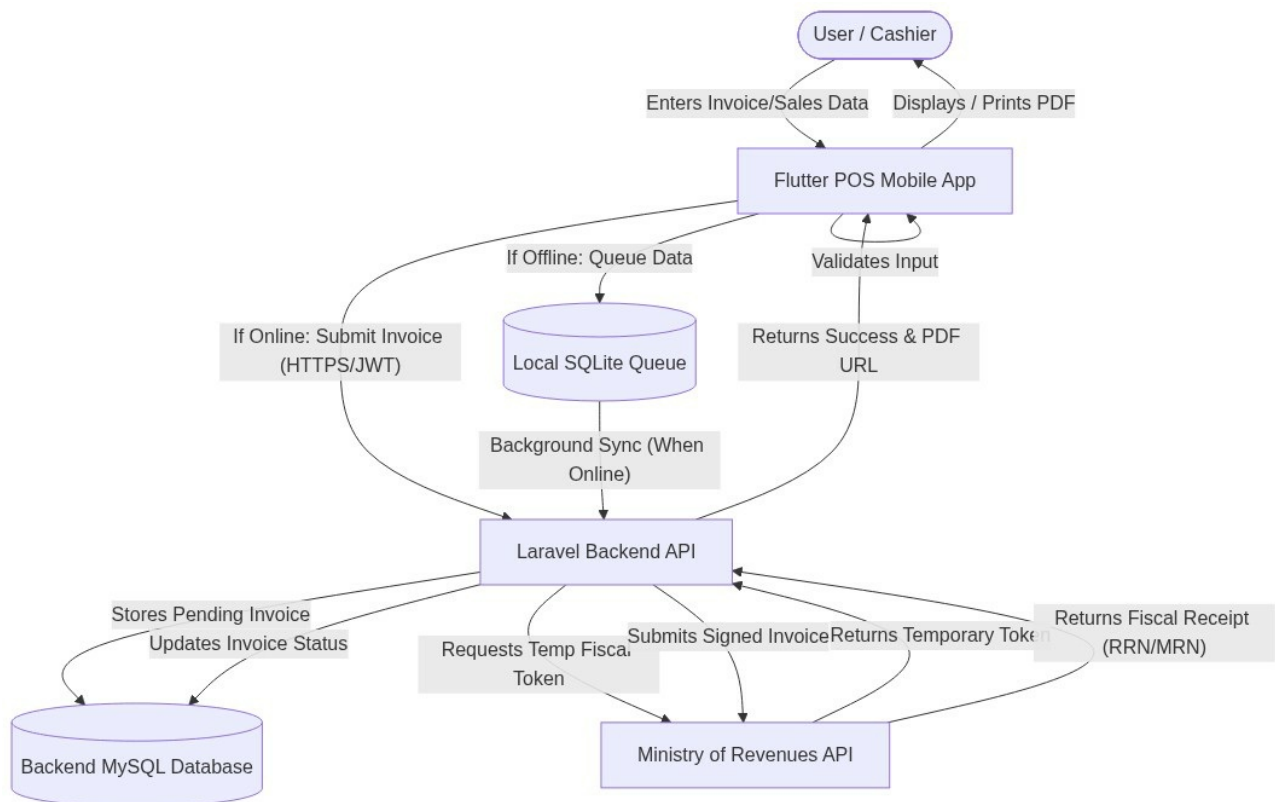
The following legal prerequisites and administrative documents are mandatory prior to the finalization of the INSA audit:

- 1. **Updated Trade License:** [Placeholder for Trade License ID]
 - 2. **TIN Number or National ID:** [Placeholder for TIN Number]
 - 3. **System/Product Patent Certificate (if available):** [Placeholder for Patent]
-

3. Technical Documentation for Web Application Security Testing

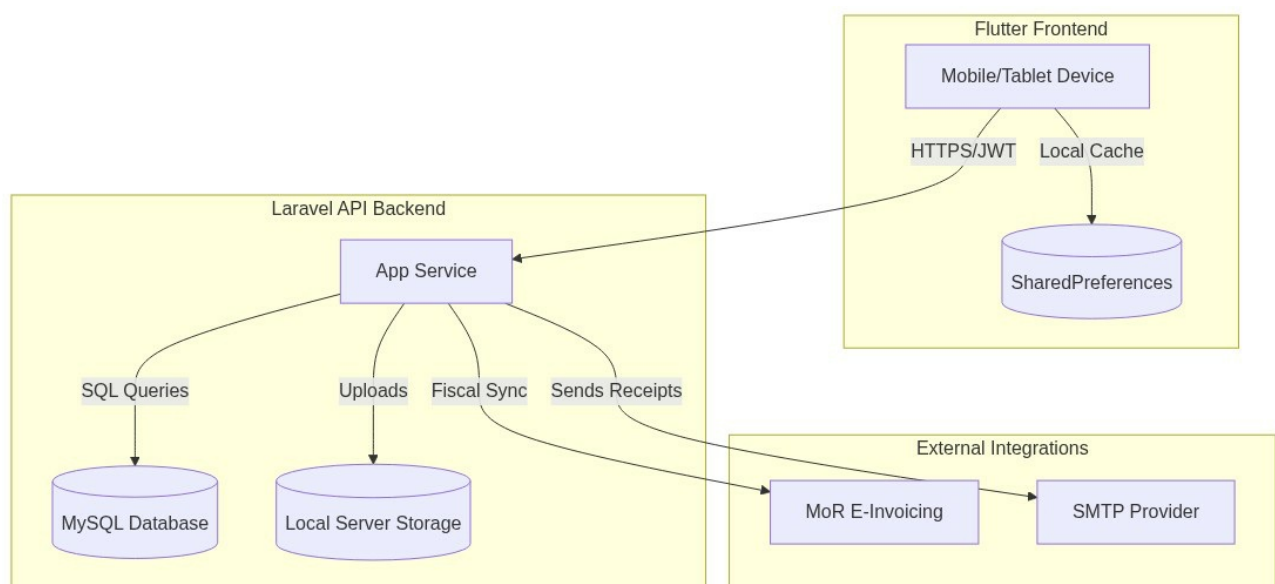
3.1 Business Architecture and Design

3.1.1 Data Flow Diagram (DFD)



1. **Invoice Initialization:** The cashier (User) selects items and initiates an invoice on the Flutter POS Application.
2. **Local Validation:** The mobile app validates inventory and necessary fields locally.
3. **Network Check & Queuing:**
 - Offline: The transaction is queued securely in local SharedPreferences.
 - Online: The request is transmitted via HTTPS to the Laravel backend.
4. **Backend Processing & Authentication:** The Laravel API authenticates the user via a JWT token. It checks the specific Branch TIN and fetches the Branch Password to request a temporary MoR token.
5. **Fiscal Registration:** The backend submits the payload to the MoR API. The MoR returns a fiscal receipt number.
6. **Finalization:** The backend updates the database and returns a success response.

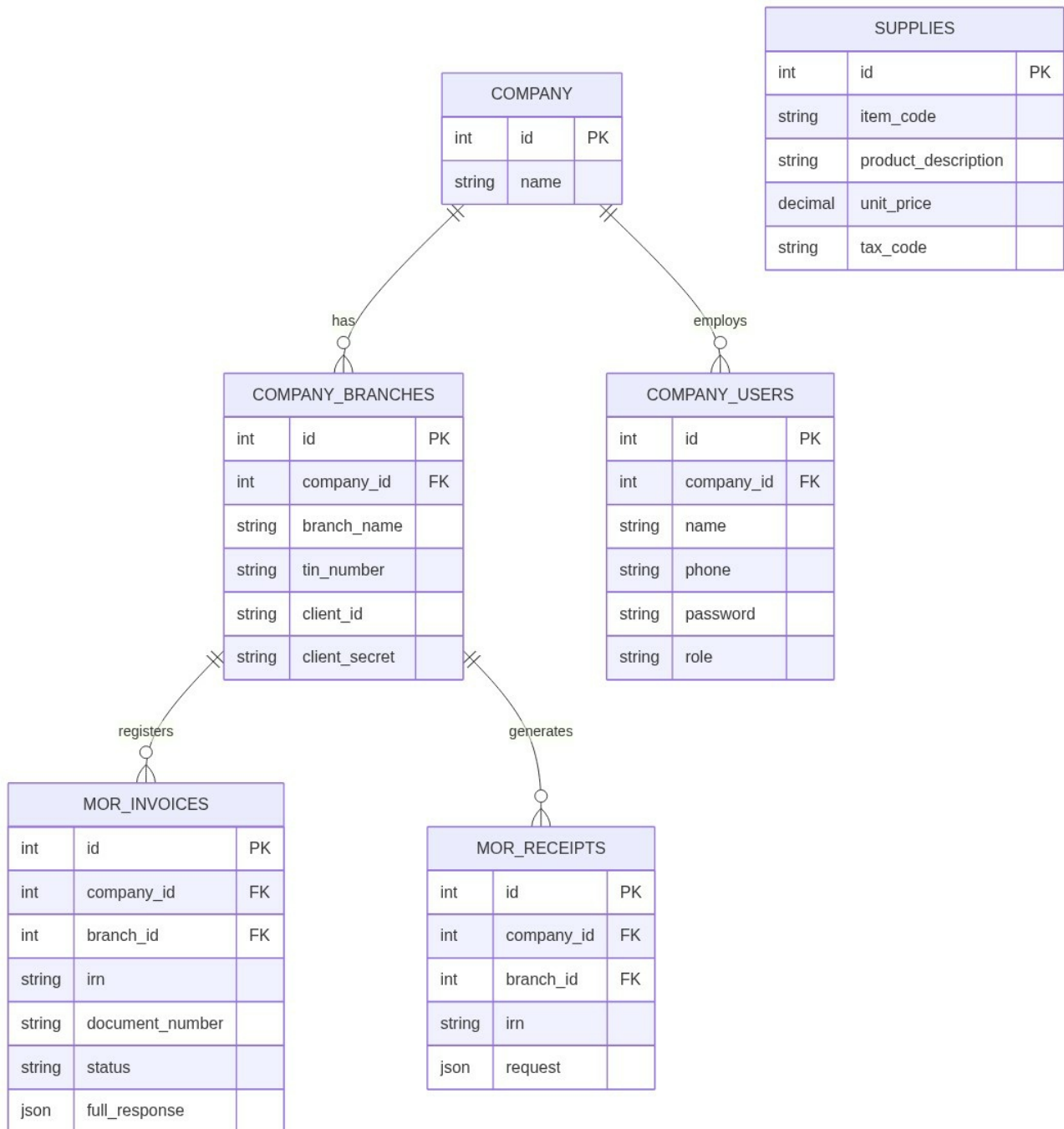
3.1.2 System Architecture Diagram



- **Presentation Layer (Frontend):** Flutter (Dart). Employs GetX for state management and routing, and SharedPreferences for offline queuing.
- **Application Layer (Backend):** Built with the Laravel 8 framework.

- **Data Layer (Backend):** Relational MySQL database. Storage for receipts is offloaded to the local server storage.
- **Integration Layer:** Connects securely with the Ministry of Revenues (MoR) API.

3.1.3 Entity Relationship Diagram (ERD)



Key Entities and Definitions:

- **COMPANY & COMPANY_USERS:** **COMPANY** represents the master tenant. **COMPANY_USERS** contains name, phone, password, and role.
- **COMPANY_BRANCHES:** Represent the physical stores tied to a company. They contain the critical tin_number, client_id, and client_secret (encrypted via model casts).
- **MOR_INVOICES:** The central fiscal transactional record. Holds irn, document_number, status, and the full_response (JSON). Tied to both a Branch and a Company.
- **MOR_RECEIPTS:** Represents actual payments synced with the MoR.
- **SUPPLIES:** Represents inventory items, holding item_code, unit_price, and tax_code.

3.2 Features of the Web Application

- **Development Frameworks:** Flutter (SDK ^3.12.2) and Dart (>=3.0.0) for Frontend; Laravel 8 (PHP) for Backend.

- **Offline Reliability:** Invoices generated during internet outages are converted to JSON and queued locally in SharedPreferences.
- **Actor Types:**
 - *Super Admin / Owner:* Global oversight, company registration.
 - *Branch Users:* Restricted to specific branch checkout functions.

3.3 Security Functionality Document

3.3.1 Authentication and Access Control

1. **Access Allocation:** Users are assigned roles (e.g., owner) upon creation.
2. **API Authentication:** Executed via JSON Web Tokens (JWT). The payload is signed with an HS256 cryptographic secret.

3.3.2 User ID Authentication

Authentication utilizes unique Phone Numbers and Passwords tied to specific Company IDs.

3.3.3 Change Password

The application relies on users to set their password during registration or account creation. There is currently no mechanism that forces a user to change their password upon their first login.

3.3.4 User ID / Password Management

Each user registers with a unique Phone Number. The system relies on administrative procedures to discourage the sharing of accounts between cashiers.

3.3.5 Sharing Information

Users are expected to keep their credentials confidential.

3.3.6 Complexity of Password

The backend API enforces a minimum password length of six (6) characters during registration and login. Passwords are mathematically hashed exclusively using Laravel's native Bcrypt implementation (`Hash::make`). There are no programmatic requirements for special characters, numbers, or mixed casing.

3.3.7 Password Expiry

The application does not currently enforce password expiration or rotation. Passwords remain valid indefinitely unless manually changed by the user or an administrator.

3.3.8 Password History

Password history is not tracked in the database. Users are permitted to reuse old passwords if they choose to change them.

3.3.9 Invalid Login Attempt

If a user submits an incorrect password, the API returns a standard 401 Unauthorized response. The application currently relies on the hosting provider's infrastructure to handle mass brute-force attacks, as API-level rate limiting (`ThrottlesLogins`) is not explicitly implemented on the authentication endpoints.

3.3.10 Lockout Duration

Because invalid login attempts are not systematically counted in the database, there is no automatic lockout duration enforced on user accounts.

3.3.11 Session Idle Time

The JWT Time-To-Live (TTL) is configured to expire after 60 minutes. Once the 60 minutes elapse, the token is invalidated and the user must re-authenticate.

3.3.12 Encryption & Secure Communication

- **Data in Transit:** The application relies on standard TLS encryption provided by the hosting environment for all API communications.
- **Data at Rest:** Passwords are never stored in plaintext (hashed using Bcrypt). Sensitive MoR credentials (`client_secret`, `api_key`) in the `company_branches` table are encrypted at rest using Laravel's model casting mechanism (`encrypted`).

3.4 Secure Coding Standard Documentation

3.4.1 Secure Coding Requirements

- **Input Validation:** API endpoints utilize Laravel Validator logic (e.g., `Validator::make`) to ensure required fields and correct data types are passed.
- **Database Safety:** The Eloquent ORM is utilized to interact with the MySQL database, which utilizes PDO parameterization to mitigate SQL Injection (SQLi) vulnerabilities.
- **Secrets Management:** Database credentials and JWT secrets are injected via environment variables (`.env`).

3.5 Threat Modeling & Risk Management

3.5.1 Asset Identification

1. **Critical (High Risk):** MoR fiscal tokens, Branch TIN passwords, user password hashes.
2. **Important (Medium Risk):** Invoice historical data.
3. **Public (Low Risk):** Item catalogs.

3.5.2 Risk Assessment and Management

- **Spoofing:** Mitigated by stateless JWT validation ensuring requests originate from authenticated users.
- **Information Disclosure:** Prevented by hiding sensitive data (like the `client_secret` and `api_key`) from API JSON responses using Laravel's `$hidden` array on Eloquent models.

3.6 Previous Security Testing Reports

The application relies on standard Laravel and Flutter linting alongside standard functional testing. Any comprehensive penetration testing is performed prior to major production releases locally.

4. API Security Audit Requirements

4.1 API Types

The backend exposes RESTful APIs utilizing HTTP methods and JSON formats.

4.2 Authentication Mechanism

1. **Client to Backend:** Authenticated using **JWT (JSON Web Tokens)** passed in the `Authorization: Bearer <token>` header.
2. **Backend to MoR:** The Laravel server reads the branch's specific TIN and Password from the database, exchanges them for a temporary MoR token, and injects this temporary token into the headers of fiscal requests.

4.3 Third-Party Integrations

The primary integration is the **Ethiopian Ministry of Revenues (MoR) e-invoicing API**.

4.4 Authorization and Access Control

Endpoints are protected by JWT middleware to ensure that only authenticated companies and their respective users can access their financial records.